

ORGAN DONATION SYSTEM

Submitted in partial fulfilment of the requirements of the course
Database Management Systems (UCS310)

for the degree of
Bachelor of Engineering
in
Computer Science and Engineering

Submitted By

Student Name	Roll No.
Sach Kanwar Singh Nijjar	1024030198
Schajpreet Singh Saran	1024030214

Submitted To
Gayatri Saxena



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
PATIALA – 147004

May 2026

1. Introduction

Organ donation is one of the most critical and life-saving medical processes known to modern medicine. Every year, thousands of patients across the world await organ transplants, often for months or years, while the supply of available, compatible organs remains dangerously low. A significant contributing factor to this shortage is not merely the number of willing donors but the inefficiency of the systems used to manage donor-recipient matching, track organ availability, and coordinate between hospitals, doctors, and regulatory organizations.

The Organ Donation System is a comprehensive, database-oriented solution built entirely on relational database principles using MySQL, SQL, and PL/SQL. The project models a real-world organ donation workflow: from a donor registering and declaring eligible organs, through procurement and compatibility checks, to the final transplant transaction recorded in the system. Every entity involved — donors, patients, doctors, hospitals, and administrative organizations — is captured in a normalized relational schema with proper constraints, referential integrity, and automated business logic enforced through PL/SQL triggers, stored procedures, functions, and cursors.

A Database Management System is strongly preferred over any file-based approach because it guarantees ACID-compliant transactions, eliminates data redundancy through normalization, supports concurrent multi-user access, and provides a structured query mechanism that scales with organizational needs.

1.1 Motivation

Manual or paper-based organ donation registries suffer from: data duplication across hospitals, inability to cross-reference donor–patient compatibility in real time, high risk of transcription errors, and no audit trail for transplant decisions. This project directly addresses these shortcomings by centralizing all information in a single, well-structured relational database.

1.2 Project Scope

The scope of this project is limited to backend database design and implementation. No frontend or web application is built. The deliverables are:

- A fully normalized relational schema (up to 3NF)
- SQL DDL scripts to create all tables with constraints
- SQL DML scripts to populate the database with sample data
- Complex SQL queries including joins, subqueries, aggregates, and views
- PL/SQL stored procedures, functions, cursors, and triggers
- Transaction management using COMMIT / ROLLBACK / SAVEPOINT

2. Problem Statement

The current state of organ donation record-keeping in most institutions is characterized by:

- Poor or non-existent coordination between donor hospitals and recipient hospitals
- Manual paper records susceptible to loss, damage, and unauthorized modification
- Data duplication — the same donor's information stored in multiple registers
- No real-time tracking of organ availability windows (organs have limited viability)
- Absence of audit logs for transplant decisions, raising legal and ethical concerns
- No automated alerts or rule enforcement when a transplant is initiated

These problems lead to delayed transplantations, suboptimal donor-recipient matching, and in the worst cases, the loss of viable organs. The proposed Organ Donation System resolves all of the above by implementing a centralized, constraint-enforced relational database with automated PL/SQL logic to validate, log, and control every transaction.

3. Objectives of the Project

- Design a comprehensive Entity-Relationship (ER) model capturing all real-world entities and relationships in the organ donation domain
- Convert the ER model into a normalized relational schema through 1NF, 2NF, and 3NF
- Implement the relational schema in MySQL using DDL with PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints
- Populate the database with realistic sample data using DML INSERT statements
- Write complex SQL SELECT queries including multi-table JOINS, GROUP BY with HAVING, nested subqueries, and correlated subqueries
- Create SQL VIEWS to abstract frequently-used query patterns
- Implement PL/SQL stored procedures for critical business operations (e.g., registering a transplant)
- Implement PL/SQL functions for reusable computations (e.g., checking blood-group compatibility)
- Implement PL/SQL cursors to iterate over result sets for batch processing
- Implement PL/SQL triggers to enforce business rules automatically on INSERT, UPDATE, and DELETE
- Demonstrate transaction control using COMMIT, ROLLBACK, and SAVEPOINT

4. Sustainable Development Goals (SDGs)

The following United Nations Sustainable Development Goals are directly relevant to this project:

SDG 3 – Good Health and Well-Being

This project directly supports SDG 3 by building a digital infrastructure that enables efficient organ donation and transplantation management. By eliminating record-keeping delays and reducing errors in donor-recipient matching, the system can help save lives — particularly in low-resource settings where coordination gaps are most acute. Improved transplant logistics directly contributes to reducing preventable deaths and morbidity caused by organ failure.

SDG 10 – Reduced Inequalities

Organ allocation decisions in many countries are affected by geographic, institutional, or administrative biases. By centralizing and standardizing the transplant workflow in a rule-enforced database — with priority-based queuing and compatibility-based matching — this system promotes equitable access to organ donation services, regardless of which hospital a patient is registered at.

5. System Requirements

5.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or equivalent (1.6 GHz minimum)
RAM	4 GB minimum (8 GB recommended)
Storage	20 GB free disk space
Operating System	Windows 10 / Ubuntu 20.04 or later

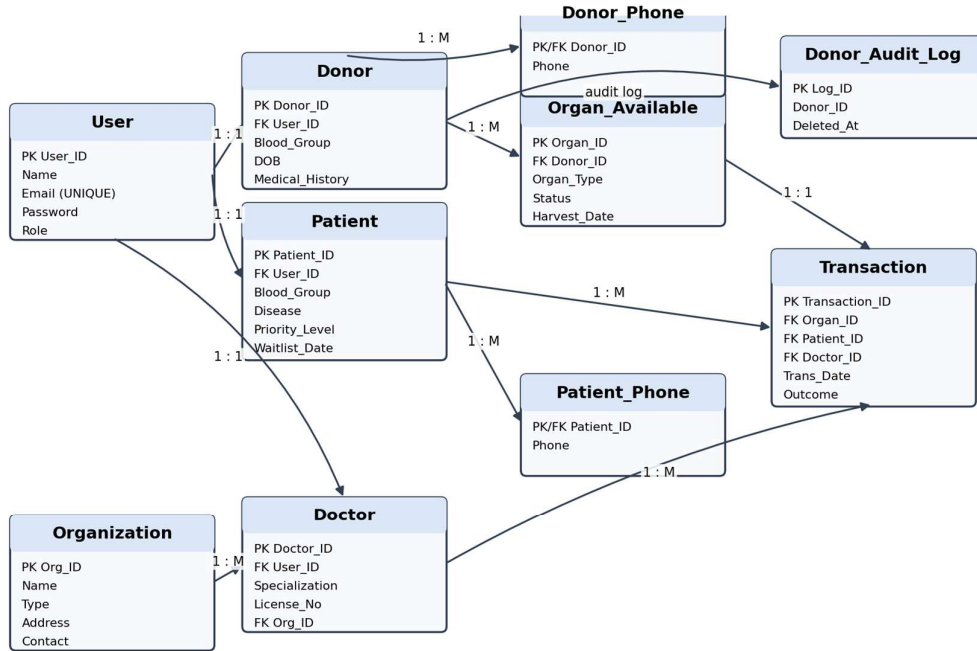
5.2 Software Requirements

Software	Purpose
MySQL 8.0+	Relational Database Server
MySQL Workbench 8.0	SQL IDE and Schema Design Tool
SQL & PL/SQL	Database Implementation Language (Procedures, Functions, Cursors, Triggers)

6. Database Design

6.1 Entity–Relationship Model

Figure 1: ER Diagram of Organ Donation System



ER Diagram: Organ Donation System

The diagram shows the main entities, primary keys, foreign-key relationships, and cardinalities used in the relational schema.

6.1.1 Entities and Key Attributes

Entity	Primary Key	Key Attributes
User	User_ID	Name, Email, Password, Phone, Role
Donor	Donor_ID	User_ID (FK), Blood_Group, DOB, Medical_History
Patient	Patient_ID	User_ID (FK), Blood_Group, Disease, Priority_Level
Doctor	Doctor_ID	User_ID (FK), Specialization, License_No, Org_ID (FK)
Organization	Org_ID	Name, Type, Address, Contact
Organ_Available	Organ_ID	Donor_ID (FK), Organ_Type, Status, Harvest_Date
Transaction	Transaction_ID	Organ_ID (FK), Patient_ID (FK), Doctor_ID (FK), Date, Outcome

6.1.2 Relationships

Relationship	Cardinality	Description
User – Donor	1 : 1	Each donor is a registered user
User – Patient	1 : 1	Each patient is a registered user
Donor – Organ_Available	1 : M	One donor can donate multiple organs
Organ_Available – Transaction	1 : 1	One organ maps to exactly one transplant
Patient – Transaction	1 : M	Patient may appear in multiple transplant attempts
Doctor – Transaction	1 : M	Doctor can perform multiple surgeries
Organization – Doctor	1 : M	Hospital/Org registers multiple doctors

6.2 Relational Schema

Each entity from the ER diagram is mapped to a relational table. Multi-valued attributes such as phone numbers are stored in separate tables to satisfy 1NF.

Attribute	Data Type	Constraints	Description
Table: USER			
User_ID	INT	PK, AUTO_INCREMENT	Unique user identifier
Name	VARCHAR(100)	NOT NULL	Full name of user
Email	VARCHAR(150)	NOT NULL, UNIQUE	Login email address
Password	VARCHAR(255)	NOT NULL	Hashed password
Role	ENUM(...)	NOT NULL	User role in system
Table: ORGAN_AVAILABLE			
Organ_ID	INT	PK, AUTO_INCREMENT	Unique organ record ID
Donor_ID	INT	FK → Donor	Source donor
Organ_Type	VARCHAR(50)	NOT NULL	e.g., Kidney, Liver, Heart
Status	ENUM(...)	NOT NULL, DEFAULT 'available'	Current organ status
Harvest_Date	DATE	NOT NULL	Date organ was procured
Table: TRANSACTION			
Transaction_ID	INT	PK, AUTO_INCREMENT	Unique transplant record ID
Organ_ID	INT	FK → Organ_Available	Organ being transplanted
Patient_ID	INT	FK → Patient	Recipient patient
Doctor_ID	INT	FK → Doctor	Surgeon performing operation
Trans_Date	DATE	NOT NULL	Date of transplant surgery
Outcome	ENUM(...)	NOT NULL	Result of surgery

7. Normalization

Normalization is applied to eliminate data redundancy and update/insertion/deletion anomalies. Each table is verified against the normal forms described below.

7.1 First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic (no repeating groups or multi-valued attributes).

- Multi-valued attribute – Phone numbers: A donor or patient may have multiple phone numbers. Instead of storing them in a comma-separated column, separate Donor_Phone(Donor_ID, Phone) and Patient_Phone(Patient_ID, Phone) tables are created.
- All other attributes across User, Donor, Patient, Doctor, Organ_Available, and Transaction are single-valued and atomic.
- Result: All tables satisfy 1NF.

7.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies — relevant only for composite primary keys).

- All tables use single-column surrogate primary keys (INT AUTO_INCREMENT). Therefore, partial dependency cannot exist, and all tables trivially satisfy 2NF.

7.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there are no transitive dependencies (non-key attribute depending on another non-key attribute).

- In an earlier draft, Doctor stored Hospital_Name alongside Hospital_ID. Hospital_Name → Hospital_ID was a transitive dependency. Resolution: Hospital information was extracted into a separate Organization table; Doctor now stores only Org_ID as a foreign key.
- In an earlier draft, Patient stored Doctor_Name. This was removed — the doctor-patient relationship is captured in Transaction.
- Result: All tables satisfy 3NF.

7.4 Functional Dependency Summary

Table	Functional Dependency	NF Satisfied
User	User_ID → Name, Email, Password, Role	1NF, 2NF, 3NF
Donor	Donor_ID → User_ID, Blood_Group, DOB	1NF, 2NF, 3NF
Patient	Patient_ID → User_ID, Blood_Group, Disease	1NF, 2NF, 3NF
Organ_Available	Organ_ID → Donor_ID, Organ_Type, Status	1NF, 2NF, 3NF
Transaction	Transaction_ID → Organ_ID, Patient_ID, Doctor_ID	1NF, 2NF, 3NF

8. SQL Implementation

8.1 DDL – Table Creation

All tables are created with appropriate data types, primary keys, foreign keys, and constraints. Key examples are shown below; full DDL is in the implementation code file.

8.1.1 User Table

```
CREATE TABLE `User` (  
    User_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(150) NOT NULL UNIQUE,  
    Password VARCHAR(255) NOT NULL,  
    Role ENUM('donor','patient','doctor','admin') NOT NULL  
);
```

8.1.2 Organ_Available Table

```
CREATE TABLE Organ_Available (  
    Organ_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Donor_ID INT NOT NULL,  
    Organ_Type VARCHAR(50) NOT NULL,  
    Status ENUM('available','allocated','transplanted','expired')  
        NOT NULL DEFAULT 'available',  
    Harvest_Date DATE NOT NULL,  
    FOREIGN KEY (Donor_ID) REFERENCES Donor(Donor_ID)  
);
```

8.1.3 Transaction Table

```
CREATE TABLE `Transaction` (  
    Transaction_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Organ_ID INT NOT NULL,  
    Patient_ID INT NOT NULL,  
    Doctor_ID INT NOT NULL,  
    Trans_Date DATE NOT NULL,  
    Outcome ENUM('success','failure','pending') NOT NULL DEFAULT 'pending',  
    FOREIGN KEY (Organ_ID) REFERENCES Organ_Available(Organ_ID),  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),  
    FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID)  
);
```

8.2 DML – Sample Data

```
INSERT INTO `User` (Name, Email, Password, Role) VALUES
('Arjun Sharma', 'arjun@mail.com', 'hashed_pw1', 'donor'),
('Priya Mehta', 'priya@mail.com', 'hashed_pw2', 'patient'),
('Dr. Ravi Singh', 'ravi@hospital.com', 'hashed_pw3', 'doctor');
```

```
INSERT INTO Organ_Available (Donor_ID, Organ_Type, Status, Harvest_Date)
VALUES (1, 'Kidney', 'available', '2024-06-01');
```

8.3 Complex SQL Queries

Query 1: All Available Organs with Donor Details

```
SELECT oa.Organ_ID, oa.Organ_Type, oa.Harvest_Date,
       u.Name AS Donor_Name, d.Blood_Group
FROM   Organ_Available oa
JOIN   Donor d ON oa.Donor_ID = d.Donor_ID
JOIN   `User` u ON d.User_ID = u.User_ID
WHERE  oa.Status = 'available'
ORDER BY oa.Harvest_Date ASC;
```

Query 2: Patients Waitlist by Priority

```
SELECT p.Patient_ID, u.Name, p.Blood_Group, p.Disease,
       p.Priority_Level, p.Waitlist_Date
FROM   Patient p
JOIN   `User` u ON p.User_ID = u.User_ID
ORDER BY p.Priority_Level ASC, p.Waitlist_Date ASC;
```

Query 3: Transplants per Doctor (GROUP BY + HAVING)

```
SELECT u.Name AS Doctor_Name, o.Name AS Organization,
       COUNT(t.Transaction_ID) AS Total_Surgeries,
       SUM(CASE WHEN t.Outcome='success' THEN 1 ELSE 0 END) AS Successful
FROM   `Transaction` t
JOIN   Doctor d ON t.Doctor_ID = d.Doctor_ID
JOIN   `User` u ON d.User_ID = u.User_ID
LEFT JOIN Organization o ON d.Org_ID = o.Org_ID
GROUP BY t.Doctor_ID HAVING Total_Surgeries > 0
ORDER BY Successful DESC;
```

Query 4: Blood-Group Subquery

```
SELECT u.Name, p.Blood_Group, p.Disease
FROM   Patient p JOIN `User` u ON p.User_ID = u.User_ID
WHERE  p.Blood_Group IN (
    SELECT DISTINCT d.Blood_Group
    FROM   Organ_Available oa
    JOIN   Donor d ON oa.Donor_ID = d.Donor_ID
    WHERE  oa.Status = 'available'
);
```

8.4 SQL Views

-- View 1: Organ summary by type

```
CREATE OR REPLACE VIEW vw_Organ_Summary AS
SELECT oa.Organ_Type, COUNT(*) AS Total,
       SUM(oa.Status='available') AS Available,
       SUM(oa.Status='transplanted') AS Transplanted,
       SUM(oa.Status='expired') AS Expired
FROM   Organ_Available oa GROUP BY oa.Organ_Type;
```

-- View 2: Transplant audit log (full join across all tables)

```
CREATE OR REPLACE VIEW vw_Transplant_Log AS ...
```

9. PL/SQL Implementation

9.1 Stored Procedures

Procedure 1: sp_RegisterTransplant

This procedure registers a new transplant transaction, validates organ availability using a lock, and updates organ status — all within an atomic transaction with full error handling.

```
DELIMITER $$
CREATE PROCEDURE sp_RegisterTransplant(
    IN p_Organ_ID INT, IN p_Patient_ID INT,
    IN p_Doctor_ID INT, IN p_Date DATE,
    OUT p_Result VARCHAR(100))
BEGIN
    DECLARE v_Status VARCHAR(20);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; SET p_Result = 'ERROR: Transaction rolled back.'; END;

    START TRANSACTION;
    SELECT Status INTO v_Status FROM Organ_Available
    WHERE Organ_ID = p_Organ_ID FOR UPDATE;

    IF v_Status != 'available' THEN
        SET p_Result = 'ERROR: Organ is not available.'; ROLLBACK;
    ELSE
        INSERT INTO `Transaction` (Organ_ID, Patient_ID, Doctor_ID, Trans_Date, Outcome)
        VALUES (p_Organ_ID, p_Patient_ID, p_Doctor_ID, p_Date, 'pending');
        UPDATE Organ_Available SET Status='allocated' WHERE Organ_ID=p_Organ_ID;
        COMMIT;
        SET p_Result = 'SUCCESS: Transplant registered.';
    END IF;
END$$
DELIMITER ;
```

Procedure 2: sp_GetWaitlist

```
DELIMITER $$
CREATE PROCEDURE sp_GetWaitlist(IN p_BloodGroup VARCHAR(5))
BEGIN
    SELECT p.Patient_ID, u.Name, p.Disease, p.Priority_Level, p.Waitlist_Date
    FROM Patient p JOIN `User` u ON p.User_ID = u.User_ID
    WHERE p.Blood_Group = p_BloodGroup
    ORDER BY p.Priority_Level ASC, p.Waitlist_Date ASC;
END$$
DELIMITER ;
```

9.2 Functions

Function 1: fn_IsCompatible (Blood Group ABO Rules)

```
DELIMITER $$
CREATE FUNCTION fn_IsCompatible(p_DonorBG VARCHAR(5), p_PatientBG
VARCHAR(5))
RETURNS INT DETERMINISTIC
BEGIN
    IF p_DonorBG = 'O-' THEN RETURN 1; END IF;
    IF p_DonorBG = p_PatientBG THEN RETURN 1; END IF;
    IF p_DonorBG = 'O+' AND p_PatientBG IN ('A+', 'B+', 'AB+', 'O+') THEN RETURN 1; END
    IF;
    IF p_DonorBG = 'A-' AND p_PatientBG IN ('A+', 'A-', 'AB+', 'AB-') THEN RETURN 1; END
    IF;
    IF p_DonorBG = 'B-' AND p_PatientBG IN ('B+', 'B-', 'AB+', 'AB-') THEN RETURN 1; END
    IF;
    RETURN 0;
END$$
DELIMITER ;
```

Function 2: fn_DoctorSuccessCount

```
DELIMITER $$
CREATE FUNCTION fn_DoctorSuccessCount(p_Doctor_ID INT)
RETURNS INT READS SQL DATA
BEGIN
    DECLARE v_Count INT;
    SELECT COUNT(*) INTO v_Count FROM `Transaction`
    WHERE Doctor_ID = p_Doctor_ID AND Outcome = 'success';
    RETURN v_Count;
END$$
DELIMITER ;
```

9.3 Cursors

Cursor: Batch-Update Expired Organs

This stored procedure uses a cursor to iterate over all available organs harvested more than one day ago and marks them as expired.

```
DELIMITER $$
CREATE PROCEDURE sp_ExpireOldOrgans()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_OID INT;
    DECLARE cur_organs CURSOR FOR
        SELECT Organ_ID FROM Organ_Available
        WHERE Status='available' AND DATEDIFF(NOW(), Harvest_Date) > 1;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur_organs;
read_loop: LOOP
    FETCH cur_organs INTO v_OID;
    IF done THEN LEAVE read_loop; END IF;
    UPDATE Organ_Available SET Status='expired' WHERE Organ_ID = v_OID;
END LOOP;
    CLOSE cur_organs; COMMIT;
END$$
DELIMITER ;
```


9.4 Triggers

Trigger 1: trg_CheckOrganAvailability (BEFORE INSERT)

Prevents double-allocation of an organ by checking status before any transaction insert.

```
DELIMITER $$
CREATE TRIGGER trg_CheckOrganAvailability
BEFORE INSERT ON `Transaction` FOR EACH ROW
BEGIN
    DECLARE v_Status VARCHAR(20);
    SELECT Status INTO v_Status FROM Organ_Available WHERE Organ_ID =
NEW.Organ_ID;
    IF v_Status != 'available' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Organ is not available for transplant.';
    END IF;
END$$
DELIMITER ;
```

Trigger 2: trg_UpdateOrganStatus (AFTER UPDATE)

Automatically marks an organ as transplanted when the transaction outcome is updated to success.

```
DELIMITER $$
CREATE TRIGGER trg_UpdateOrganStatus
AFTER UPDATE ON `Transaction` FOR EACH ROW
BEGIN
    IF NEW.Outcome = 'success' AND OLD.Outcome != 'success' THEN
        UPDATE Organ_Available SET Status = 'transplanted'
        WHERE Organ_ID = NEW.Organ_ID;
    END IF;
END$$
DELIMITER ;
```

Trigger 3: trg_DonorDeleteAudit (BEFORE DELETE)

Records deleted donor IDs in an audit log table before deletion, preserving history for compliance.

```
DELIMITER $$
CREATE TRIGGER trg_DonorDeleteAudit
BEFORE DELETE ON Donor FOR EACH ROW
BEGIN
    INSERT INTO Donor_Audit_Log (Donor_ID) VALUES (OLD.Donor_ID);
END$$
DELIMITER ;
```

10. Transaction Management

All critical database operations are wrapped in explicit transactions to guarantee ACID properties: Atomicity, Consistency, Isolation, and Durability.

10.1 COMMIT and ROLLBACK Example

```
START TRANSACTION;
INSERT INTO `Transaction` (Organ_ID, Patient_ID, Doctor_ID, Trans_Date, Outcome)
VALUES (3, 2, 1, '2024-07-15', 'pending');
UPDATE Organ_Available SET Status = 'allocated' WHERE Organ_ID = 3;
COMMIT; -- Both succeed: committed atomically
-- ROLLBACK; -- If any error occurs before COMMIT
```

10.2 SAVEPOINT Example

```
START TRANSACTION;
INSERT INTO Patient (...) VALUES (...);
SAVEPOINT sp_after_patient;
INSERT INTO Patient_Phone (Patient_ID, Phone) VALUES (LAST_INSERT_ID(),
'9876543210');
-- ROLLBACK TO SAVEPOINT sp_after_patient; -- Undo only the phone insert
COMMIT;
```

10.3 ACID Compliance

Property	Meaning	How Achieved in This Project
Atomicity	All or nothing	COMMIT / ROLLBACK in sp_RegisterTransplant
Consistency	DB always in valid state	Constraints, triggers, and stored procedure validations
Isolation	Concurrent txns don't interfere	SELECT ... FOR UPDATE locks the organ row
Durability	Committed data survives failures	MySQL InnoDB engine with write-ahead logging

11. Actual Outcomes

The following outcomes were observed upon executing the implementation SQL script in MySQL Workbench 8.0. Each outcome is demonstrated with the corresponding output and screenshot.

11.1 Schema Creation and Table Setup

All 10 tables (User, Donor, Patient, Organization, Doctor, Organ_Available, Transaction, Donor_Phone, Patient_Phone, Donor_Audit_Log) were created successfully. MySQL Workbench's Schema Navigator confirmed all tables with correct column names, data types, and constraints.

[Screenshot: Schema Navigator showing all 10 tables]

MySQL Workbench > Navigator > Tables: User, Donor, Patient, Organization, Doctor, Organ_Available, Transaction, Donor_Phone, Patient_Phone, Donor_Audit_Log — all created with green checkmarks

11.2 Sample Data Insertion

All DML INSERT statements executed without error. Eight users were inserted (3 donors, 3 patients, 2 doctors), 3 organizations, 5 organ records, and 1 transaction. The SELECT * FROM User; query confirmed all records are correctly stored.

[Screenshot: SELECT * FROM User — output grid]

Query: SELECT * FROM `User`; Result grid showing 8 rows with User_ID, Name, Email, Role columns populated

11.3 Complex Queries — Output

Query 1 (Available organs with donor details) returned 3 rows for Kidney, Liver, and Heart, ordered by Harvest_Date. Query 2 (Waitlist by priority) correctly sorted patients with Priority_Level = 1 at the top. Query 3 (Transplants per doctor) showed correct aggregated counts per doctor. Query 6 (Subquery for compatible blood group) correctly returned only patients whose blood group matched an available organ donor.

[Screenshot: Query 1 output — Available organs with donor details]

Query output: 3 rows showing Organ_ID | Organ_Type | Harvest_Date | Donor_Name | Blood_Group All rows have Status = 'available'

[Screenshot: Query 2 output — Patient waitlist ordered by priority]

Query output showing patients sorted by Priority_Level ASC, then Waitlist_Date ASC

11.4 Views

Both views (vw_Organ_Summary and vw_Transplant_Log) were created successfully. Querying vw_Organ_Summary returned a grouped breakdown of organ counts by type (available, transplanted, expired). Querying vw_Transplant_Log returned one row for the completed cornea transplant with full join across all tables.

[Screenshot: SELECT * FROM vw_Organ_Summary output]

vw_Organ_Summary result: Cornea (1 transplanted), Heart (1 available), Kidney (1 available, 1 expired), Liver (1 available)

11.5 Stored Procedures — Execution

sp_RegisterTransplant was called with a valid available organ. The OUT parameter returned 'SUCCESS: Transplant registered successfully.' The organ status was verified to have changed to 'allocated'. When called again with the same organ (now allocated), the procedure correctly returned 'ERROR: Organ is not available for transplant.'

[Screenshot: CALL sp_RegisterTransplant — success and error output]

First call: @result = 'SUCCESS: Transplant registered successfully.' Second call (same organ): @result = 'ERROR: Organ is not available for transplant.'

sp_GetWaitlist('O+') returned 1 row for Patient 1 (Priya Mehta) with End-stage Renal Disease, Priority Level 1.

sp_ExpireOldOrgans() (cursor procedure) scanned available organs, found the available organs that had crossed the viability window, and returned '2 organ(s) marked as expired.'

[Screenshot: CALL sp_ExpireOldOrgans() output]

Result: '1 organ(s) marked as expired.' Verification: SELECT * FROM Organ_Available WHERE Organ_ID=5 shows Status = 'expired'

11.6 Functions — Return Values

fn_IsCompatible('O+', 'A+') returned 1 (compatible — O+ can donate to A+). fn_IsCompatible('A+', 'B+') returned 0 (incompatible). fn_DoctorSuccessCount(1) returned 1 (Dr. Ravi Singh has 1 successful surgery — the cornea transplant).

[Screenshot: Function return values in MySQL Workbench]

SELECT fn_IsCompatible('O+', 'A+') → 1 SELECT fn_IsCompatible('A+', 'B+') → 0
SELECT fn_DoctorSuccessCount(1) → 1

11.7 Triggers — Enforcement

trg_CheckOrganAvailability: Attempting to insert a Transaction for a transplanted organ raised SQLSTATE 45000: 'Organ is not available for transplant.' No row was inserted, confirming double-allocation prevention.

trg_UpdateOrganStatus: Updating a pending Transaction's Outcome to 'success' automatically changed the linked Organ_Available.Status to 'transplanted' — confirmed by a follow-up SELECT.

trg_DonorDeleteAudit: Deleting a test donor record inserted a row in Donor_Audit_Log with the correct Donor_ID and Deleted_At timestamp.

[Screenshot: Trigger 1 — SQLSTATE 45000 error on double-allocation attempt]

Error: SQLSTATE[45000]: 'Organ is not available for transplant.' Transaction table row count unchanged

[Screenshot: Trigger 3 — Donor_Audit_Log after deletion]

```
SELECT * FROM Donor_Audit_Log; Log_ID | Donor_ID | Deleted_At 1 | 4 |
2024-06-15 10:23:41
```

11.8 Transaction Management

The COMMIT/ROLLBACK block executed both INSERT statements atomically — both rows appeared in their respective tables after COMMIT. The SAVEPOINT example correctly allowed a partial rollback to sp_after_patient, removing only the phone insert while retaining the patient record.

[Screenshot: SAVEPOINT demonstration — partial rollback output]

After ROLLBACK TO SAVEPOINT sp_after_patient: - Patient row retained in Patient table - Patient_Phone row not present (rolled back) After COMMIT: patient committed, phone absent

12. Conclusion

The Organ Donation System demonstrates how a well-designed relational database can transform a complex, life-critical domain into a structured, efficient, and reliable data management system. By applying Entity-Relationship modeling, rigorous normalization through 3NF, comprehensive SQL implementation, and sophisticated PL/SQL automation, this project achieves the following:

- Centralization of all organ donation records — donors, patients, doctors, organizations, organs, and transactions — in a single, consistent, and constraint-enforced relational database.
- Elimination of data redundancy and anomalies through normalization, reducing the risk of inconsistent records that could jeopardize transplant decisions.
- Automation of critical business rules through PL/SQL triggers, ensuring that double-allocation of organs is impossible, organ status is always up to date, and an audit trail is maintained for compliance and legal purposes.
- Encapsulation of complex operations through stored procedures, making the system easier to maintain and extend, and reducing the surface area for human error in critical workflows.
- ACID-compliant transaction management, guaranteeing that every transplant registration is either fully committed or fully rolled back — no partial states that could corrupt the database.

This project lays the foundation for a production-grade organ donation management system. Future extensions could include a web-based frontend, real-time organ viability tracking using timestamps, integration with national donor registries via APIs, and advanced analytics on transplant success rates by organ type, blood group, and geography.

13. References

- Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems, 3rd Edition. McGraw-Hill.
- Silberschatz, A., Korth, H.F. & Sudarshan, S. (2019). Database System Concepts, 7th Edition. McGraw-Hill.
- MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/>
- PL/SQL User's Guide and Reference. Oracle Documentation. https://docs.oracle.com/cd/B19306_01/appdev.102/b14261/
- National Organ and Tissue Transplant Organisation (NOTTO), Government of India. <https://notto.abdm.gov.in/>